

Time & Space complexity analysis report:

Given two strings S and T,

DP formulation:

$dp[i][j]$ = min cost to transform first i characters of S into first j characters of T

Recurrence Relation:

$$dp[i][j] = \begin{cases} dp[i-1][j-1], & \text{if } S[i-1] = T[j-1] \\ \min \begin{cases} dp[i-1][j-1] + R & \text{(Replace)} \\ dp[i-1][j] + D & \text{(Delete)} \\ dp[i][j-1] + I & \text{(Insert)} \end{cases} & \text{if } S[i-1] \neq T[j-1] \end{cases}$$

Base Case initialization:

$dp[0][j] = j \cdot I$ for (insert all characters of T into empty S)

$dp[i][0] = i \cdot D$ for (delete all characters of S to get empty T)

Time Complexity:

For dp table we iterate over a table of size $(n+1) * (m+1)$.

So for dp table, $O(n*m)$.

For operations backtracking, $O(n+m)$.

So finally, $O(n*m)$.

Space Complexity:

For dp table, a 2D array is used. so $O(n*m)$

For operations storing, $O(n+m)$.

So finally space complexity $O(n*m)$.